

MINISTRY OF EDUCATION AND RESEARCH



TECHNICAL UNIVERSITY
OF CLUJ-NAPOCA, ROMANIA

Assignment 3

Software Design

Author: Maria Moruțan

Group: 30433

FACULTY OF AUTOMATION AND COMPUTER SCIENCE

May 2026

Contents

1	Problem definition	2
1.1	Users and roles	2
2	Architecture	2
2.1	Microservices decomposition	2
2.2	Inter-service communication	3
2.3	Internal service structure	3
3	System diagrams	3
3.1	Use case diagram	3
3.2	Entity-relationship diagram	5
3.3	Class diagram	6
3.4	Sequence diagram	7
4	Functional requirements	8
4.1	All functionality from Assignments 1 & 2	8
4.2	Microservices separation	8
4.3	Command-Query Responsibility Segregation (CQRS)	8
5	Design patterns	8
5.1	Template pattern	8
5.2	Command pattern	9
5.3	Observer pattern	9
5.4	Singleton pattern	9
6	Implementation details	9
6.1	Authentication with JWT	9
6.2	Cross-service data access	10
6.3	Asynchronous notifications	10
7	Unit testing	10

1 Problem definition

This assignment extends the Pet Management System developed in Assignments 1 and 2 by switching to a microservices-based architecture. The domain remains unchanged — a pet shop system supporting pets, items, users, exports, and notifications — but the internal structure has been fully decomposed into independent, deployable services that communicate over HTTP/REST.

The main objectives are to improve scalability, modularity, and maintainability by separating each major responsibility into its own service with its own database, and to introduce CQRS, the Command and Template design patterns, and comprehensive unit testing.

1.1 Users and roles

The system continues to support the same roles defined in previous assignments:

- **Visitor** – can view pets and items but cannot modify data.
- **Administrator** – has full access to all system functionalities, including CRUD operations on all entities. Receives notifications upon any CRUD operation.
- **Seller** – can add pets and items but cannot update or delete them.

Authentication is now handled by a dedicated **auth-service** using JWT tokens. Role-based access control is enforced in the **frontend-service** based on the token's claims.

2 Architecture

2.1 Microservices decomposition

The application is decomposed into six independent microservices, each with a single responsibility and its own database:

- **pet-service** (port 8081) – manages pet entities. Has its own **pet_db** MySQL database. Exposes REST endpoints for CRUD operations, search, and sorting.
- **item-service** (port 8082) – manages item entities. Has its own **item_db** MySQL database. References pets by ID only, fetching pet details from **pet-service** via HTTP when needed.
- **auth-service** (port 8083) – handles user registration, login, and JWT token generation and validation. Has its own **auth_db** MySQL database.
- **notification-service** (port 8084) – receives event notifications from other services via HTTP POST and sends email notifications. Has no database.
- **export-service** (port 8085) – fetches data from **pet-service** and **item-service** and exports it in JSON, CSV, or XML format. Has no database.
- **frontend-service** (port 8080) – serves the Thymeleaf HTML UI and acts as a gateway, calling the appropriate backend services for each user action. Has no database.

2.2 Inter-service communication

Services communicate using synchronous HTTP/REST via Spring's `RestTemplate`. Each service that depends on another has a dedicated client class (e.g., `PetClient`, `ItemClient`, `AuthClient`) that encapsulates the HTTP calls and handles failures gracefully.

The cross-service dependency graph is as follows:

- `frontend-service` calls `auth-service`, `pet-service`, `item-service`, and `export-service`.
- `item-service` calls `pet-service` (to resolve pet names) and `notification-service`.
- `pet-service` calls `notification-service`.
- `export-service` calls `pet-service` and `item-service`.
- `notification-service` and `auth-service` call no other services.

2.3 Internal service structure

Each service follows a clear internal layered structure:

- **Controller layer** – exposes REST endpoints, handles HTTP requests and responses.
- **Service layer** – contains business logic, implements CQRS separation.
- **Repository layer** – data access using Spring Data JPA (where applicable).
- **Domain layer** – JPA entities and DTOs.
- **Client layer** – HTTP clients for calling other services.
- **Event layer** – Observer pattern components for event-driven communication.

3 System diagrams

3.1 Use case diagram

The use case diagram illustrates the interactions between system actors and the main functionalities. The three roles interact differently based on their permissions. Authentication is now handled by `auth-service` and enforced across all services via JWT tokens.

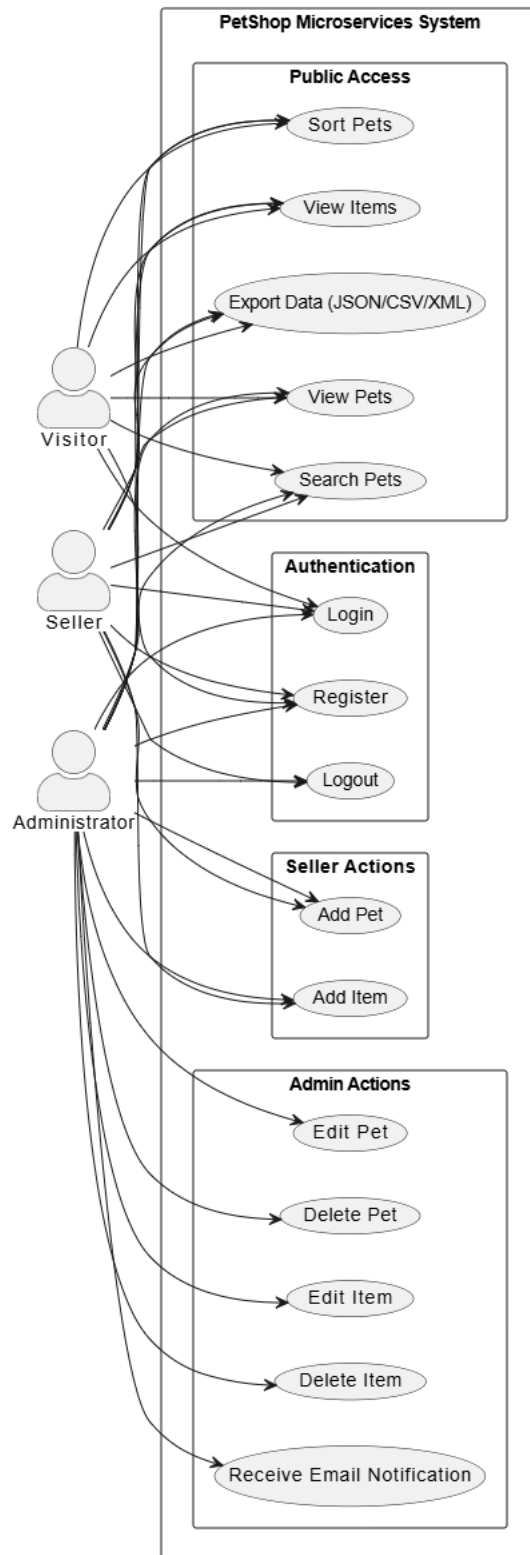


Figure 1: Use-case diagram for the Pet Management System (microservices)

3.2 Entity-relationship diagram

Each service has its own isolated database. The `pet_db` contains only the `pets` table. The `item_db` contains the `items` table, which stores `pet_id` as a plain integer foreign reference (no JPA join). The `auth_db` contains the `users` table.

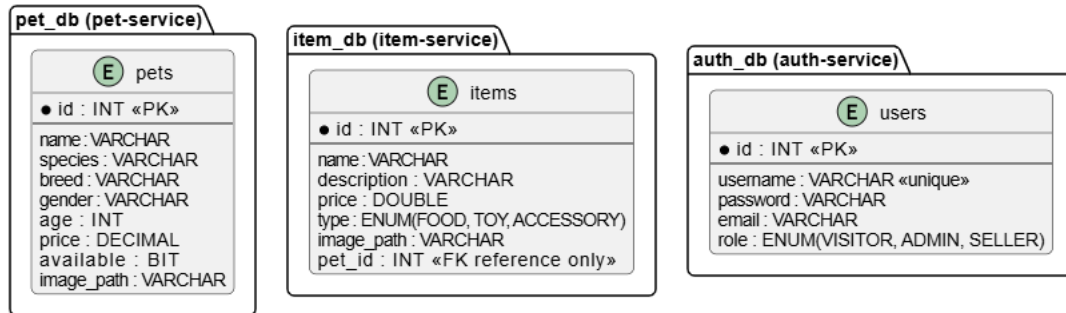


Figure 2: Entity-relationship diagram — separate databases per service

3.3 Class diagram

The class diagram illustrates the core components of each service, including controllers, services, repositories, entities, DTOs, client classes, and event handling components.

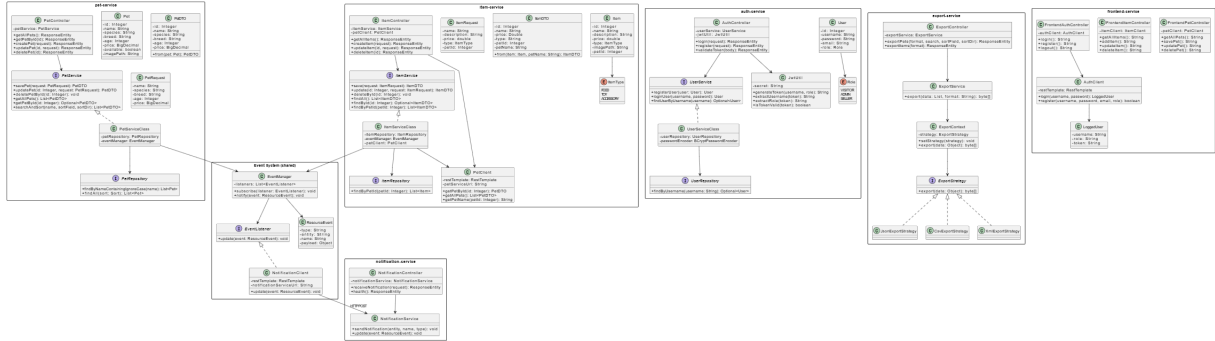


Figure 3: Class diagram of the microservices system

3.4 Sequence diagram

The sequence diagram illustrates the full workflow of creating a new pet: the user submits the form in `frontend-service`, which calls `pet-service`, which saves the pet, fires an event, and asynchronously notifies `notification-service`.

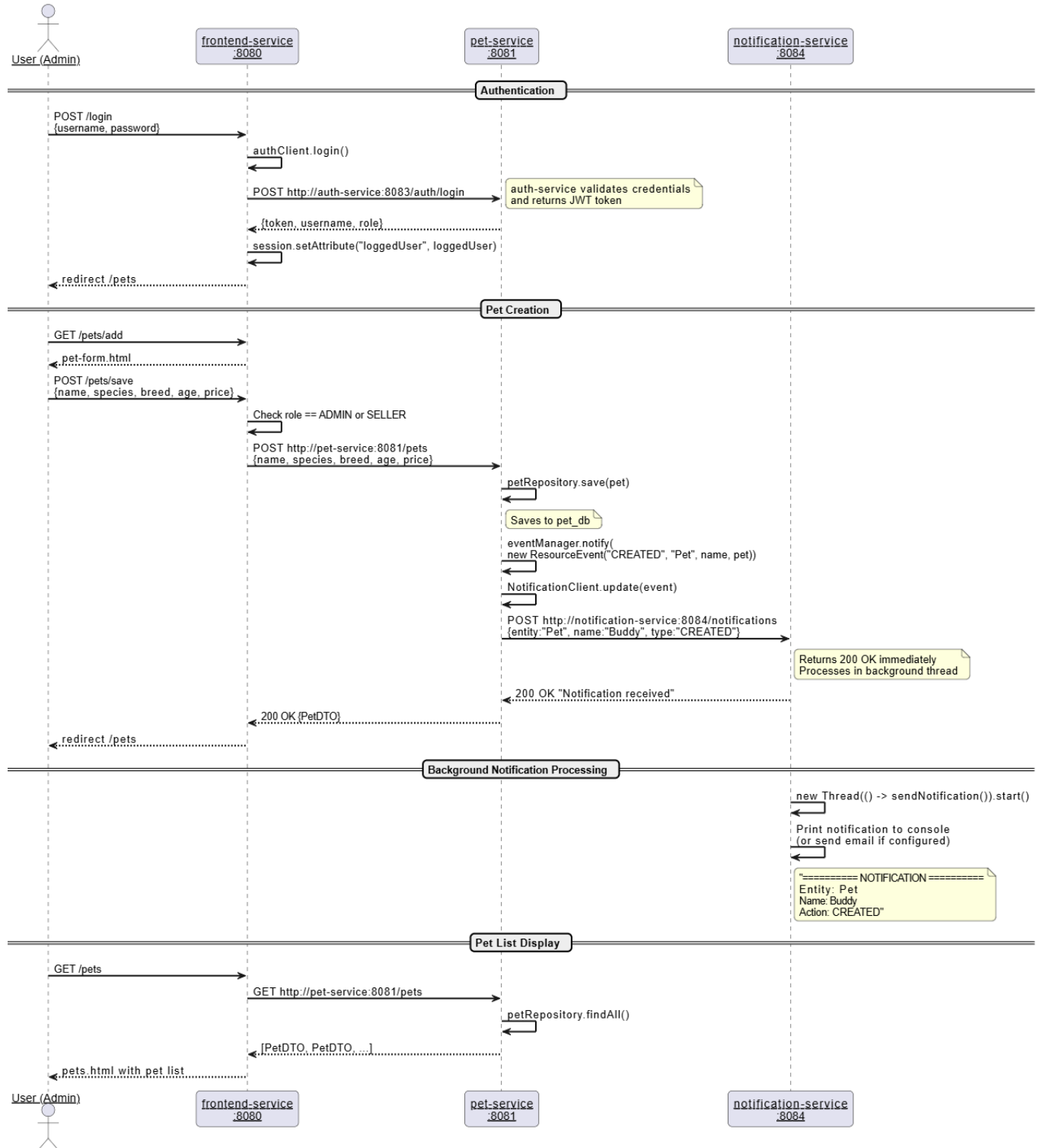


Figure 4: Sequence diagram for pet creation across microservices

4 Functional requirements

4.1 All functionality from Assignments 1 & 2

All existing functionality is preserved: CRUD operations on pets and items, user authentication and registration, role-based access control, search and sorting, export in multiple formats, and email notifications. The behavior from the user's perspective is identical — only the internal architecture has changed.

4.2 Microservices separation

Each major responsibility is implemented as an independent microservice with a clear, well-defined boundary:

- Pet management is fully encapsulated in `pet-service`.
- Item management is fully encapsulated in `item-service`. The `@ManyToOne Pet pet` JPA relationship has been replaced with a plain `Integer petId` field, eliminating the cross-service dependency at the database level.
- Authentication and user management are fully encapsulated in `auth-service`.
- Notification logic is fully encapsulated in `notification-service`.
- Export logic is fully encapsulated in `export-service`.
- The UI layer is fully encapsulated in `frontend-service`.

4.3 Command-Query Responsibility Segregation (CQRS)

Each service separates its operations into two distinct types:

- **Command operations** – create, update, or delete data. For example, `savePet(PetRequest)`, `updatePet(Integer, PetRequest)`, `deletePetById(Integer)`.
- **Query operations** – read, search, or filter data. For example, `getAllPets()`, `getPetById(Integer)`, `searchAndSort(String, String, String)`.

This separation is reflected in the `PetService` and `ItemService` interfaces, where command methods accept request DTOs and return response DTOs, while query methods return read-only DTOs without modifying state. A dedicated `PetRequest` and `ItemRequest` DTO is used for write operations, while `PetDTO` and `ItemDTO` are used for read responses.

5 Design patterns

5.1 Template pattern

The Template pattern is applied in the export system. The `ExportStrategy` interface defines the contract for exporting data, while three concrete implementations provide format-specific behavior:

- `JsonExportStrategy` – serializes data using Jackson’s `ObjectMapper`.
- `CsvExportStrategy` – manually constructs CSV output with headers and rows.
- `XmlExportStrategy` – produces XML output.

The `ExportContext` class selects the appropriate strategy at runtime based on the requested format. The `ExportService` exposes a unified `export(data, format)` method that delegates to the context, abstracting the format-specific details from the controller layer.

5.2 Command pattern

The Command pattern is applied to encapsulate UI-initiated actions. Each user action in the frontend (create, update, delete) is represented as an HTTP request with a structured body (`PetRequest`, `ItemRequest`) sent to the appropriate service. The service layer then executes the command independently of the UI event that triggered it, decoupling the frontend from the backend implementation.

For example, clicking “Save” on the Add Pet form sends a `POST /pets` request with a `PetRequest` body to `pet-service`. The `PetController` receives the command and delegates to `PetServiceClass.savePet()`, which executes the operation and fires a `ResourceEvent`.

5.3 Observer pattern

The Observer pattern is implemented via `EventManager` and `EventListener` in each service. When a pet or item is created, updated, or deleted, the service emits a `ResourceEvent`. The `NotificationClient` (which implements `EventListener`) receives the event and forwards it to `notification-service` via HTTP POST. The notification-service then processes the event asynchronously in a background thread to avoid blocking the calling service.

5.4 Singleton pattern

Spring’s `@Service`, `@Component`, and `@Repository` annotations ensure that all service beans are singletons within their application context. The `EventManager`, `RestTemplate`, and all client classes are managed as Spring beans, guaranteeing a single shared instance per service.

6 Implementation details

6.1 Authentication with JWT

The `auth-service` implements JWT-based authentication using the `jjwt` library. Upon successful login, a signed JWT token is generated containing the user’s username and role. The token is stored in the `frontend-service` HTTP session and can be validated by any service via `POST /auth/validate`.

6.2 Cross-service data access

Since services have independent databases, cross-service data access is handled by HTTP calls rather than JPA joins. For example, when `item-service` needs to return a pet's name alongside an item, it calls `GET /pets/{id}` on `pet-service` and assembles the result into an `ItemDTO`. All client calls are wrapped in try-catch blocks so that a failure in one service does not crash another.

6.3 Asynchronous notifications

The `notification-service` processes incoming events asynchronously using a background thread (`new Thread(() -> ...).start()`). This ensures that the calling service (e.g., `pet-service`) receives an immediate 200 OK response and is not blocked waiting for the email or log operation to complete.

7 Unit testing

Each service includes unit tests covering both command and query operations using JUnit 5 and Mockito. Dependencies such as repositories, event managers, and HTTP clients are mocked so tests run independently of any running services or databases.

The following test classes are provided:

- `PetServiceTest` – tests `savePet`, `updatePet`, `deletePetById`, `getAllPets`, `getPetById`, `searchPets`.
- `ItemServiceTest` – tests `save`, `update`, `deleteById`, `findAll`, `findById`, `findByPetId`.
- `ExportServiceTest` – tests JSON, CSV, and XML export for both pets and items, including error handling for invalid formats.
- `AuthServiceTest` – tests user registration with password encoding, login with valid and invalid credentials, and JWT token generation and validation.
- `NotificationServiceTest` – tests notification sending and graceful failure when the mail sender throws an exception.